

CO3-AT3-Debugging and Optimization

View Only

[← Back](#)

QUESTIONS

Q1ATM Withdrawal – Exception
Handling Debugging
10 marks**Q2**Hospital Registration –
NullPointerException
Debugging
10 marks**Q3**E-Commerce Product Search –
Array Exception Debugging
10 marks**Q4**Bank Account – Race Condition
Debugging
10 marks**Q5**Railway Reservation –
Synchronization Debugging

Q5 Railway Reservation – Synchronization Debugging

10 marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class Reservation {  
  
    int seats = 1;  
  
    void bookSeat(String customer) {  
  
        if (seats > 0) {
```

10 marks

5/5 answered

```
System.out.println(customer
+ " is booking a seat");

try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Thread interrupted");
}

seats--;

System.out.println(customer
+ " booking successful");

} else {
    System.out.println(customer
+ " - No seat available");
}
}

class Customer extends Thread {

    Reservation reservation;
    String customerName;

    Customer(Reservation reservation, String customerName) {
        this.reservation = reservation;
        this.customerName = customerName;
    }
}
```

```
public void run() {  
    reservation.bookSeat(customerName);  
}  
}  
  
class RailwayReservation {  
  
    public static void main(String[] args) {  
  
        Reservation reservation = new Reservation();  
  
        Customer c1 =  
            new Customer(reservation, "Customer 1");  
  
        Customer c2 =  
            new Customer(reservation, "Customer 2");  
  
        c1.start();  
        c2.start();  
    }  
}
```

The corrected program must protect the shared seat resource using synchronization.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 class Reservation{  
2     int seats=1;  
3     synchronized void bookSeat(String customer) {
```

Test Input

```

4         if(seats>0){
5             System.out.println(customer+" is booking a seat");
6             try{
7                 Thread.sleep(100);
8             }catch(InterruptedException e){
9                 System.out.println("Thread interrupted");
10            }
11            seats--;
12            System.out.println(customer+" booking successful");
13        }else{
14            System.out.println(customer+" - No seat available");
15        }
16    }
17    public static void main(String[] args){
18        Reservation reservation=new Reservation();
19        Customer c1=new Customer(reservation,"Customer 1");
20        Customer c2=new Customer(reservation,"Customer 2");
21        c1.start();
22        c2.start();
23    }
24 }
25 class Customer extends Thread{
26     Reservation reservation;
27     String customerName;
28     Customer(Reservation reservation,String customerName){
29         this.reservation=reservation;
30         this.customerName=customerName;
31     }
32     public void run(){
33         reservation.bookSeat(customerName);
34     }
35 }

```

stdin input (optional)

Output

Customer 1 is booking a seat
Customer 1 booking successful
Customer 2 - No seat available

⌚ Pending manual review

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.